



รายงานทางวิชาการ

เรื่อง

AI-Assisted Testing vs. Automatic Test Case Generation Algorithms: A Benchmark and Test Coverage Evaluation

จัดทำโดย

673380415-5	นายพัชรพล	กองแก้ว
673380430-9	นายอนันต์เอกก์	ใหญ่พงศกร
673380425-2	นายวงศธร	ธน.ยอด

อาจารย์ประจำวิชา: ผศ.ดร.ชิตสุธา สุ่มเล็ก

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353201 Software Quality Assurance

ภาคเรียน 1 ปีการศึกษา 2569 สาขาวิทยาการคอมพิวเตอร์

วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

บทคัดย่อ

โครงการนี้ศึกษาการเปรียบเทียบประสิทธิภาพของเทคนิคการสร้างกรณีทดสอบซอฟต์แวร์อัตโนมัติสำหรับโปรแกรมภาษา Java ระหว่างอัลกอริทึม Automatic Test Generation และ Generative Artificial Intelligence โดยใช้ Defects4J เป็น Benchmark หลักของการทดลอง ภายใต้แนวคิดว่าการประเมินระบบสร้าง Test Case ไม่ควรพิจารณาเฉพาะ Code Coverage แต่ควรพิจารณาความสำเร็จของการสร้างและประเมิน Test Case ความสามารถในการตรวจพบข้อบกพร่อง และต้นทุนในการสร้างร่วมกัน

การทดลองเปรียบเทียบ 4 เทคนิค ได้แก่ DynaMOSA ผ่าน EvoSuite, Reanimator ภายใน KEX, Gemini และ Claude โดย DynaMOSA เป็น Dynamic Many-Objective Sorting Algorithm สำหรับ Search-Based Test Generation ส่วน Reanimator เป็นอัลกอริทึมแบบ backward search ที่เอกสาร KEX เรียกว่า “original backward-search based algorithm” และใช้เพื่อแปลง symbolic inputs หรือ object representations ให้เป็นลำดับการเรียก public API และ executable JUnit test cases (Abdullin et al., 2021) ขณะที่รายงานการแข่งขัน SBST 2021 เรียก Reanimator ว่า “a novel backward search algorithm” (Panichella et al., 2021) สำหรับ Gemini และ Claude ใช้ Prompt ที่กำหนดร่วมกับ Source Code และ Dependency Context เพื่อสร้าง JUnit 4 Test Suite โดยอัตโนมัติ

ชุดข้อมูลของการทดลองประกอบด้วยข้อบกพร่อง 854 bugs จาก 17 projects ใน Defects4J กระบวนการเตรียมข้อมูลพบ Target Entries 1,073 รายการ และหลังจากตัด 3 รายการที่ไม่พบ Source Code และ 3 รายการที่ไม่ใช่ Java resource ออก เหลือ Java runnable targets จำนวน 1,067 targets สำหรับกระบวนการสร้าง Test Case

ผลการทดลองพบว่า DynaMOSA ผ่าน EvoSuite มี Evaluation Success สูงที่สุด 825 จาก 854 bugs หรือ 96.604% รองลงมาคือ Reanimator ผ่าน KEX 684 bugs หรือ 80.094%, Gemini 485 bugs หรือ 56.792% และ Claude 322 bugs หรือ 37.705% เมื่อพิจารณา Fault Detection พบว่า Gemini ตรวจพบข้อบกพร่องได้มากที่สุด 56 bugs หรือ 6.557% รองลงมาคือ Claude 19 bugs หรือ 2.225%, Reanimator ผ่าน KEX 6 bugs หรือ 0.703% และ DynaMOSA ผ่าน EvoSuite 2 bugs หรือ 0.234%

ในด้าน Coverage เมื่อพิจารณาเฉพาะกรณีที่วัด Coverage สำเร็จ Gemini มี Mean Line Coverage สูงที่สุด 85.799% และ Mean Condition Coverage 80.545% ขณะที่ DynaMOSA ผ่าน EvoSuite มี Coverage Success Rate สูงที่สุด 798 จาก 854 bugs หรือ 93.443% เมื่อเปรียบเทียบบน Common Evaluation Set 200 bugs Gemini ยังคงมี Fault Detection Rate สูงที่สุด 10.5% และบน Common Coverage Set 177 bugs Gemini มี Mean Line Coverage 91.073% และ Mean Condition Coverage 86.648%

สำหรับ Generative AI พบว่า Gemini ใช้ Total Tokens ตามค่าที่ระบบ API รายงาน 14,918,091 tokens และตรวจพบ 56 bugs คิดเป็น 3.7538 detected bugs ต่อหนึ่งล้าน tokens ขณะที่ Claude ใช้ 25,954,456 tokens และตรวจพบ 19 bugs หรือ 0.7321 detected bugs ต่อหนึ่งล้าน tokens ภายใต้ Configuration ของการทดลองนี้

โดยสรุป ไม่มีเทคนิคใดที่ดีที่สุดในทุกมิติ DynaMOSA ผ่าน EvoSuite เด่นด้านความเร็วและความครอบคลุมของ Benchmark, Reanimator ผ่าน KEX มี Evaluation Success สูงและสร้าง Candidate Tests ได้จำนวนมาก, Gemini เด่นด้าน Fault Detection และ Coverage ในกรณีที่ประเมินสำเร็จ ส่วน Claude แม้มี Success Rate ต่ำกว่า Gemini แต่ยังมี Fault Detection และ Coverage ที่สูงกว่า Automatic Test Generation บางเทคนิคในหลายมิติ ผลการศึกษาจึงสนับสนุนให้ประเมิน Test Generation Technique ด้วยตัวชี้วัดหลายด้านร่วมกัน

คำสำคัญ — Software Testing, Automatic Test Case Generation, DynaMOSA, EvoSuite, KEX, Reanimator, Generative AI, Gemini, Claude, Defects4J, Test Coverage, Fault Detection

คำนำ

รายงานโครงงานเรื่อง “AI-Assisted Testing vs. Automatic Test Case Generation Algorithms: A Benchmark and Test Coverage Evaluation” จัดทำขึ้นเป็นส่วนหนึ่งของรายวิชา CP353201 Software Quality Assurance โดยมีวัตถุประสงค์เพื่อศึกษาและเปรียบเทียบประสิทธิภาพของอัลกอริทึมสำหรับการสร้างกรณีทดสอบซอฟต์แวร์อัตโนมัติและ Generative AI ภายใต้ชุดข้อมูลและกระบวนการประเมินเดียวกัน

การศึกษานี้เปรียบเทียบ 4 เทคนิค ได้แก่ DynaMOSA ผ่านเครื่องมือ EvoSuite, Reanimator ซึ่งเป็นอัลกอริทึม backward search ที่ใช้ภายใน KEX, Gemini และ Claude โดยผู้จัดทำมุ่งให้ความสำคัญกับทั้งความสำเร็จในการสร้างและประเมิน Test Case, Fault Detection, Line Coverage, Condition Coverage และ Token Efficiency ของ Generative AI

โครงงานใช้ Defects4J เป็น Benchmark ครอบคลุม 854 bugs จาก 17 projects และจัดทำ Benchmark Protocol, Docker Environment, Scripts, Prompt, Configuration, Raw Results, Generated Test Code, Evaluation Results และ Coverage Results เพื่อให้สามารถตรวจสอบและทำการทดลองได้

ผู้จัดทำหวังว่ารายงานฉบับนี้จะช่วยให้ผู้อ่านเข้าใจทั้ง Search-Based Test Generation, Symbolic/Concolic Analysis, อัลกอริทึม Reanimator และการประยุกต์ใช้ Generative AI ในงาน Software Testing รวมถึงเห็นความแตกต่างระหว่าง Coverage, Executability และ Fault Detection จากผลการทดลองจริง

ท้ายที่สุดนี้ ผู้จัดทำขอขอบพระคุณ ผู้ช่วยศาสตราจารย์ ดร.จิตสุธา สุ่มเล็ก อาจารย์ประจำรายวิชา CP353201 Software Quality Assurance ที่ได้ให้คำแนะนำ แนวทาง และข้อเสนอแนะตลอดการดำเนินโครงงาน ตลอดจนสมาชิกในกลุ่มทุกคนที่ร่วมกันศึกษา พัฒนา ทดลอง วิเคราะห์ และรวบรวมผลจนโครงงานสำเร็จลุล่วง

ผู้จัดทำ

นายพัชรพล กองแก้ว

นายอนันต์เอกก์ ใหญ่พงศกร

นายวงศธร ธน.ยอด

สารบัญ

บทคัดย่อ.....	ก
คำนำ.....	ค
สารบัญ	ง
สารบัญตาราง	ช
สารบัญภาพ	ฅ
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ.....	1
1.2 วัตถุประสงค์	2
1.3 ขอบเขตของการศึกษา	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ.....	3
1.5 นิยามศัพท์ที่เกี่ยวข้อง	3
บทที่ 2 ทฤษฎีและงานที่เกี่ยวข้อง.....	5
2.1 การทดสอบซอฟต์แวร์.....	5
2.2 การสร้างกรณีทดสอบโดยอัตโนมัติ.....	5
2.3 Search-Based Software Testing	5
2.4 EvoSuite	5
2.5 DynaMOSA	6
2.6 Symbolic Execution	6
2.7 KEX และอัลกอริทึม Reanimator.....	6
2.8 Generative AI และ LLM สำหรับการสร้าง Test Case	7
2.9 Gemini.....	7
2.10 Claude	7
2.11 Defects4J.....	7

สารบัญ (ต่อ)

2.12 Test Coverage	8
2.13 Fault Detection และ Bug-Revealing Test	8
2.14 ความสัมพันธ์ระหว่าง Coverage และ Fault Detection	8
2.15 งานวิจัยที่เกี่ยวข้อง	8
บทที่ 3 วิธีดำเนินการและการออกแบบการทดลอง	10
3.1 ภาพรวมการดำเนินงาน	10
3.2 ชุดข้อมูลและ Benchmark.....	10
3.3 การกำหนด Target Classes	11
3.4 สภาพแวดล้อมในการทดลอง	11
3.5 การสร้าง Test Case ด้วย DynaMOSA ผ่าน EvoSuite	12
3.6 การสร้าง Test Case ด้วย Reanimator ผ่าน KEX.....	12
3.7 การออกแบบ Prompt สำหรับ Gemini และ Claude.....	12
3.8 Configuration ของ Gemini และ Claude	13
3.9 การจัดเก็บผลการสร้าง Test Case.....	13
3.10 กระบวนการ Evaluation.....	13
3.11 หลักเกณฑ์การจำแนกผล	14
3.12 การวัด Test Coverage	14
3.13 ตัวชี้วัดที่ใช้ในการเปรียบเทียบ	14
3.14 Common Case Analysis	15
3.15 AI Token Analysis	15
3.16 Failure, Timeout และ Missing Cases.....	15
3.17 การรวบรวมผลการทดลอง.....	15
3.18 Workflow ของการทดลอง	16
3.19 การออกแบบเพื่อ Reproducibility	17

สารบัญ (ต่อ)

บทที่ 4 ผลการทดลอง	18
4.1 ภาพรวมผลการทดลอง	18
4.2 Evaluation Success.....	18
4.3 Fault Detection.....	19
4.4 Candidate Tests และ Bug-Revealing Tests.....	20
4.5 Test Coverage	20
4.6 สถานะ Coverage.....	21
4.7 Common Evaluation Set.....	21
4.8 Common Coverage Set.....	22
4.9 ผลแยกตาม Project.....	23
4.10 AI Token Analysis	24
4.11 AI Token Efficiency.....	24
4.12 สรุปผลเชิงเปรียบเทียบ	26
บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ	28
5.1 สรุปผลการดำเนินงาน.....	28
5.2 อภิปรายผลการทดลอง	28
5.3 AI Token Efficiency.....	29
5.4 ผลลัพธ์การเรียนรู้	30
5.5 ปัญหาที่พบ	30
5.6 ข้อจำกัดของการทดลอง.....	31
5.7 Threats to Validity.....	31
5.8 ข้อเสนอแนะและแนวทางพัฒนาต่อ	32
5.9 สรุปผลโครงการ.....	32
รายการอ้างอิง.....	34
ภาคผนวก ก Prompt Templates.....	37
ภาคผนวก ข Reproducibility Guide	39
ภาคผนวก ค โครงสร้างผลลัพธ์และ Configuration	41

สารบัญตาราง

ตารางที่ 3.1 จำนวน Bugs ในแต่ละ Defects4J Project.....	10
ตารางที่ 3.2 สรุปการคัดกรอง Target Entries.....	11
ตารางที่ 3.3 สภาพแวดล้อมหลักของการทดลอง	11
ตารางที่ 3.4 หลักเกณฑ์การจำแนก Candidate Test.....	14
ตารางที่ 4.1 ผลการทดลองภาพรวม.....	18
ตารางที่ 4.2 สถานะ Evaluation	18
ตารางที่ 4.3 Fault Detection.....	19
ตารางที่ 4.4 Candidate Tests และ Bug-Revealing Tests	20
ตารางที่ 4.5 ผล Test Coverage	20
ตารางที่ 4.6 สถานะ Coverage	21
ตารางที่ 4.7 Common Evaluation Set	21
ตารางที่ 4.8 Common Coverage Set.....	22
ตารางที่ 4.9 Bugs Detected แยกตาม Project.....	23
ตารางที่ 4.10 การใช้ Token ของ Gemini และ Claude	24
ตารางที่ 4.11 Token Efficiency.....	24
ตารางที่ 4.12 สรุปผลเด่นของทั้ง 4 เทคนิค	26

สารบัญภาพ

รูปที่ 3.1 Workflow ของการทดลอง.....	16
รูปที่ 4.1 Overall Fault Detection Rate	18
รูปที่ 4.2 Evaluation และ Coverage Success	19
รูปที่ 4.3 Common Evaluation Fault Detection	22
รูปที่ 4.4 Common Coverage Set	23
รูปที่ 4.5 Average Tokens per Bug	24
รูปที่ 4.6 AI Evaluation Success	25
รูปที่ 4.7 AI Fault Detection Rate	26
รูปที่ 4.8 Detected Bugs per One Million Tokens	26

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

การทดสอบซอฟต์แวร์เป็นกระบวนการสำคัญของการพัฒนาซอฟต์แวร์ เนื่องจากช่วยตรวจสอบพฤติกรรมของระบบ ค้นหาข้อผิดพลาด และเพิ่มความเชื่อมั่นว่าซอฟต์แวร์ทำงานได้ตามข้อกำหนด อย่างไรก็ตาม การออกแบบและเขียนกรณีทดสอบด้วยมนุษย์เพียงอย่างเดียวใช้เวลาและทรัพยากรจำนวนมาก โดยเฉพาะระบบที่มี Source Code ขนาดใหญ่และมีเงื่อนไขการทำงานจำนวนมาก จึงเกิดแนวทาง Automatic Test Case Generation เพื่อช่วยสร้างกรณีทดสอบโดยอัตโนมัติ

หนึ่งในแนวทางสำคัญคือ Search-Based Software Testing ซึ่งประยุกต์ Search และ Optimization Algorithms เพื่อค้นหา Test Case ที่ตอบสนอง Coverage Objectives โดย EvoSuite เป็นเครื่องมือที่ได้รับการพัฒนาสำหรับโปรแกรม Java และ DynaMOSA เป็น Dynamic Many-Objective Sorting Algorithm ที่จัดการ Coverage Objectives จำนวนมากด้วยการเลือกเป้าหมายแบบพลวัต (Fraser & Arcuri, 2011; Panichella et al., 2018)

อีกแนวทางหนึ่งคือ Symbolic/Concolic Execution ซึ่งวิเคราะห์เส้นทางการทำงานของโปรแกรม และใช้ Constraint Solving เพื่อสร้าง symbolic inputs ในโครงงานนี้ใช้ KEX เป็นเครื่องมือ และใช้ Reanimator ซึ่งเอกสารของ KEX นิยามอย่างชัดเจนว่าเป็น “original backward-search based algorithm” สำหรับค้นหาลำดับการกระทำที่สร้าง target object ผ่าน public API และสร้าง executable JUnit test cases จาก symbolic inputs (Abdullin et al., 2021) นอกจากนี้รายงาน SBST Tool Competition 2021 ยังเรียก Reanimator ว่า “a novel backward search algorithm” (Panichella et al., 2021) ดังนั้นรายงานฉบับนี้ถือ Reanimator เป็นอัลกอริทึมที่ถูกนำมาใช้งานภายใน KEX ไม่ใช่ชื่อของ KEX เอง

พร้อมกันนั้น Generative AI และ Large Language Models สามารถสร้าง Source Code จาก Context และ Prompt ได้ ทำให้สามารถนำมาประยุกต์ใช้กับ Unit Test Generation โครงงานนี้เลือก Gemini และ Claude เป็น AI-Assisting Tools และกำหนด Prompt ให้สร้าง JUnit 4 Test Suite จาก Source Code เดียวกับที่ใช้ใน Benchmark

เพื่อให้การเปรียบเทียบอ้างอิงข้อบกพร่องจริง โครงงานใช้ Defects4J ซึ่งรวบรวม Real Faults จากโปรแกรม Java และมีทั้ง Buggy และ Fixed Revisions สำหรับการทดลองแบบควบคุม (Just et al., 2014) Benchmark หลักประกอบด้วย 854 active bugs จาก 17 projects และประเมินหลายมิติ ได้แก่

Evaluation Success, Coverage Success, Candidate Tests, Bug-Revealing Tests, Fault Detection Rate และ AI Token Efficiency

การพิจารณาหลายตัวชี้วัดมีความสำคัญ เนื่องจาก Code Coverage ไม่ได้เท่ากับ Fault Detection โดยตรง งานของ Inozemtseva และ Holmes (2014) แสดงให้เห็นว่า Coverage ไม่ควรถูกใช้เป็นตัวแทนของ Test Suite Effectiveness เพียงอย่างเดียว ดังนั้นโครงการนี้จึงแยกความสำเร็จในการสร้างและรัน Test ออกจากความสามารถในการตรวจพบข้อบกพร่อง

1.2 วัตถุประสงค์

1. ศึกษาและทดลองใช้อัลกอริทึม Automatic Test Case Generation สำหรับ Java ได้แก่ DynaMOSA ผ่าน EvoSuite และ Reanimator ผ่าน KEX
2. ศึกษาและประยุกต์ใช้ Gemini และ Claude สำหรับสร้าง JUnit Test Case จาก Source Code ผ่าน Prompt ที่ออกแบบไว้
3. ดำเนินการ Benchmark กับ Defects4J จำนวน 854 bugs จาก 17 Java projects และจัดเก็บผลการทดลองอย่างเป็นระบบ
4. วัดและเปรียบเทียบ Evaluation Success, Coverage Success, Line Coverage และ Condition Coverage ของแต่ละเทคนิค
5. ประเมิน Fault Detection โดยใช้ Bug-Revealing Tests และ Fault Detection Rate
6. วิเคราะห์ Token Usage และ Token Efficiency ของ Gemini และ Claude
7. ศึกษาปัญหา ข้อจำกัด และผลลัพธ์การเรียนรู้จากการทดลองจริง
8. จัดทำ Source Code, Test Code, Prompt, Configuration, Results และ Reproducibility Guide เพื่อให้ผู้อ่านสามารถทำซ้ำการทดลองได้

1.3 ขอบเขตของการศึกษา

1. ใช้ Defects4J เป็นชุดข้อมูลหลัก จำนวน 854 active bugs จาก 17 Java projects
2. เปรียบเทียบ 4 เทคนิค ได้แก่ DynaMOSA ผ่าน EvoSuite, Reanimator ผ่าน KEX, Gemini และ Claude
3. ใช้ classes.modified เป็นข้อมูลอ้างอิง Target Classes และมี 1,067 Java runnable targets หลังการคัดกรอง
4. ใช้ Bug เป็นหน่วยหลักของ Benchmark และกำหนด denominator 854 bugs สำหรับ Overall Rates
5. ใช้ Line Coverage และ Condition Coverage ตามผลที่ Defects4J รายงาน โดยไม่ตีความ Condition เป็น Branch Coverage โดยอัตโนมัติ
6. คำนวณค่า Coverage เฉพาะกรณีที่ Coverage Run สำเร็จ และไม่แทน Failure/Timeout/Missing ด้วย 0%

7. นิยาม Bug-Revealing Test เป็น Test ที่ FAIL ใน Buggy Version และ PASS ใน Fixed Version
8. วิเคราะห์ข้อมูล Token ของ Gemini และ Claude ตามค่าที่ API บันทึกไว้
9. เก็บ Artifacts และ Scripts ที่เกี่ยวข้องไว้ใน Repository ของโครงการเพื่อการตรวจสอบและทำซ้ำ

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. เข้าใจหลักการของ Search-Based Test Generation, Symbolic/Concolic Execution และ backward-search test generation
2. เข้าใจศักยภาพและข้อจำกัดของ Generative AI สำหรับ Unit Test Generation
3. สามารถเปรียบเทียบข้อดีและข้อจำกัดของทั้ง 4 เทคนิคจากผลการทดลองภายใต้ Benchmark เดียวกัน
4. เห็นความแตกต่างระหว่าง Executability, Coverage และ Fault Detection
5. ได้ Pipeline และ Dataset ผลการทดลองที่สามารถตรวจสอบและทำซ้ำได้
6. สามารถนำผลการทดลองไปใช้เป็นแนวทางสำหรับการเลือกหรือผสม Test Generation Techniques ในอนาคต

1.5 นิยามศัพท์ที่เกี่ยวข้อง

1. Software Testing หมายถึง กระบวนการตรวจสอบและประเมินพฤติกรรมของซอฟต์แวร์เพื่อค้นหาข้อผิดพลาดและตรวจสอบความสอดคล้องกับข้อกำหนด
2. Automatic Test Case Generation หมายถึง กระบวนการสร้าง Test Case โดยอัตโนมัติด้วยเครื่องมือหรืออัลกอริทึม
3. DynaMOSA หมายถึง Dynamic Many-Objective Sorting Algorithm สำหรับ Search-Based Test Generation ซึ่งเลือก Coverage Targets แบบพลวัต (Panichella et al., 2018)
4. KEX หมายถึง เครื่องมือ Automatic White-Box Test Generation สำหรับ JVM Bytecode ที่รองรับ Symbolic และ Concolic Execution
5. Reanimator หมายถึง อัลกอริทึม backward search ที่ใช้ภายใน KEX เพื่อค้นหาลำดับการกระทำผ่าน public API สำหรับสร้าง object state และ executable test code จาก symbolic inputs (Abdullin et al., 2021)
6. Defects4J หมายถึง ฐานข้อมูลข้อบกพร่องจริงและโครงสร้างพื้นฐานสำหรับการทดลองกับโปรแกรม Java (Just et al., 2014)
7. Line Coverage หมายถึง สัดส่วนของบรรทัดที่ถูกดำเนินการจาก Test Suite เมื่อเทียบกับบรรทัดที่อยู่ภายใต้การวัด

8. Condition Coverage หมายถึง ค่าความครอบคลุมในช่อง Conditions ที่ Defects4J รายงาน ซึ่งรายงานฉบับนี้ไม่ดีความเป็น Branch Coverage โดยอัตโนมัติ
9. Bug-Revealing Test หมายถึง Candidate Test ที่ FAIL ใน Buggy Version และ PASS ใน Fixed Version ตาม Protocol ของโครงการ
10. Fault Detection Rate หมายถึง อัตราส่วนจำนวน Bugs ที่ตรวจพบต่อจำนวน Bugs ในชุดที่กำหนด
11. Reproducibility หมายถึง ความสามารถในการทำซ้ำการทดลองจากข้อมูล ขั้นตอน Environment และ Configuration ที่ระบุไว้

บทที่ 2

ทฤษฎีและงานที่เกี่ยวข้อง

2.1 การทดสอบซอฟต์แวร์

การทดสอบซอฟต์แวร์เป็นกิจกรรมสำคัญในกระบวนการพัฒนาซอฟต์แวร์ ใช้ตรวจสอบพฤติกรรม ค้นหาความผิดปกติ และประเมินความสอดคล้องกับความต้องการ มาตรฐาน ISO/IEC/IEEE 29119-1:2022 กำหนดแนวคิดทั่วไปที่ใช้กับกระบวนการ Software Testing ในบริบทที่แตกต่างกัน (ISO/IEC/IEEE, 2022)

โครงงานนี้มุ่ง Unit Testing ในระดับ Class/Method และประเมิน Generated Tests ผ่านการ Compile, Execute, Coverage และ Fault Detection เพื่อให้คุณภาพของ Test ไม่ถูกตัดสินจากจำนวน Test Case เพียงอย่างเดียว

2.2 การสร้างกรณีทดสอบโดยอัตโนมัติ

Automatic Test Case Generation ใช้ Source Code, Bytecode, Control Flow, Constraints หรือข้อมูลเชิงพฤติกรรมเพื่อสร้าง Test Inputs และ Test Sequences โดยอัตโนมัติ ปัญหาสำคัญคือพื้นที่ค้นหาของ Inputs และ Object States มีขนาดใหญ่ จึงมีการใช้ Search-Based Algorithms และ Symbolic Execution เพื่อช่วยลดพื้นที่ค้นหา

2.3 Search-Based Software Testing

Search-Based Software Testing มอง Test Generation เป็น Optimization Problem โดย Test Case หรือ Test Suite เป็น Candidate Solution และใช้ Fitness Function เช่น Approach Level, Branch Distance หรือ Coverage เพื่อชี้นำการค้นหา แนวคิดนี้เป็นส่วนหนึ่งของ Search-Based Software Engineering (Harman et al., 2012)

2.4 EvoSuite

EvoSuite เป็นเครื่องมือ Automatic Test Suite Generation สำหรับ Java ซึ่งใช้ Search-Based Optimization เพื่อสร้าง Test Suites และ Assertions สำหรับ Class ภายใต้การทดสอบ (Fraser & Arcuri, 2011) เครื่องมือสามารถกำหนด Search Budget, Coverage Criterion และ Configuration อื่นได้ และเหมาะกับการทำ Benchmark อัตโนมัติในระดับ Class

2.5 DynaMOSA

DynaMOSA เป็น Dynamic Many-Objective Sorting Algorithm ที่พัฒนาต่อจาก Many-Objective Search สำหรับ Automated Test Generation โดยเลือก Coverage Targets แบบพลวัตตาม Control Dependencies เพื่อลดจำนวน Objectives ที่ต้อง Optimize พร้อมกัน (Panichella et al.,

2018) ในโครงการนี้ DynaMOSA เป็นอัลกอริทึม Automatic Test Generation ตัวที่หนึ่ง และใช้ EvoSuite เป็นเครื่องมือที่ Implement/รันอัลกอริทึมดังกล่าว

2.6 Symbolic Execution

Symbolic Execution ใช้ค่าทางสัญลักษณ์แทนข้อมูลนำเข้า และสะสม Path Constraints ตามเส้นทางการทำงานของโปรแกรม ก่อนใช้ Constraint Solver เพื่อค้นหา Inputs ที่ทำให้เงื่อนไขเป็นจริง แนวคิดดังกล่าวมีรากฐานจากงานของ King (1976) ข้อจำกัดสำคัญคือ State Explosion และความซับซ้อนของ Constraints

2.7 KEX และอัลกอริทึม Reanimator

KEX เป็น Automatic White-Box Test Generation Tool สำหรับ Java/JVM Bytecode และใช้ Symbolic Execution กับ SMT Solver เพื่อสร้าง symbolic inputs สำหรับ Basic Blocks ที่ต้องการครอบคลุม (Abdullin et al., 2021)

ประเด็นที่สำคัญต่อโครงการนี้คือ Reanimator ไม่ใช่เพียงชื่อทั่วไปของส่วนประกอบ แต่เอกสาร KEX ระบุว่า KEX ส่ง symbolic inputs ให้ “an original backward-search based algorithm called Reanimator” เพื่อสร้าง executable JUnit test cases (Abdullin et al., 2021) และรายงาน SBST Tool Competition 2021 อธิบาย Reanimator ว่าเป็น “a novel backward search algorithm” (Panichella et al., 2021)

Reanimator เริ่มจาก object representation หรือ symbolic input และค้นย้อนกลับเพื่อหาลำดับ constructor/method/public API actions ที่สามารถสร้าง target object state จากนั้นใช้ Code Snippets ดังกล่าวเป็นส่วนหนึ่งของ JUnit tests ดังนั้นในการเปรียบเทียบเชิงแนวคิดของรายงานนี้ Reanimator คืออัลกอริทึม Automatic Test Generation ตัวที่สอง ส่วน KEX คือเครื่องมือ/แพลตฟอร์มที่ใช้ Symbolic/Concolic Analysis และเรียกใช้อัลกอริทึม Reanimator ในกระบวนการสร้าง Test

2.8 Generative AI และ Large Language Models สำหรับการสร้าง Test Case

Large Language Models สามารถสร้าง Source Code จาก Context และ Prompt ทำให้เกิดแนวทาง LLM-Based Unit Test Generation ซึ่งแตกต่างจาก Search-Based และ Symbolic Execution เพราะโมเดลสร้าง Test Code จากรูปแบบและ Context ที่เรียนรู้มา โดยไม่ได้ใช้ Fitness Search หรือ Constraint Solving แบบเดียวกับ Traditional Tools

งานวิจัยพบว่า Prompt Factors, Compilability และ Hallucination เป็นปัจจัยสำคัญต่อคุณภาพของ LLM-Generated Tests (Yang et al., 2024; Ouédraogo et al., 2024) ดังนั้นโครงการจึงแยก Generation Success, Evaluation Success และ Coverage Success ออกจากกัน

2.9 Gemini

Gemini เป็นตระกูล Generative AI ของ Google ที่รองรับการสร้างข้อความและ Code ผ่าน API (Google, n.d.) โครงการนี้ใช้ Gemini ผ่าน KGU IntelSphere API เพื่อสร้าง JUnit 4 Test Suite จาก Source Code และ Prompt ที่กำหนด และเก็บ Usage Metadata เพื่อวิเคราะห์ Token Efficiency

2.10 Claude

Claude เป็นตระกูล Generative AI ของ Anthropic ที่รองรับงานด้านภาษา Reasoning และ Code Generation (Anthropic, n.d.) โครงการนี้ใช้ Claude ผ่าน KGU IntelSphere API โดยใช้ Prompt ที่จัดโครงสร้างด้วย XML-like tags และนำ Generated Test Code เข้าสู่ Evaluation Protocol เดียวกับ Gemini

2.11 Defects4J

Defects4J เป็นฐานข้อมูล Real Faults สำหรับ Java และมีคำสั่ง checkout, compile, test, coverage และ export สำหรับการทำการทดลอง Controlled Experiments (Just et al., 2014) Buggy Revision ใช้ suffix b และ Fixed Revision ใช้ suffix f ซึ่งเอื้อต่อการตรวจสอบว่า Generated Test เปิดเผย Fault ที่แก้ไขแล้วได้หรือไม่

2.12 Test Coverage

Test Coverage วัดสัดส่วนของโครงสร้างโปรแกรมที่ถูกกระตุ้นจาก Test Suite โครงการนี้ใช้ Line Coverage และ Condition Coverage ตามผลของ Defects4J และคำนวณค่าเฉลี่ยเฉพาะกรณี Coverage Run สำเร็จ เพื่อแยกความล้มเหลวของ Pipeline ออกจาก Coverage ที่มีค่า 0

2.13 Fault Detection และ Bug-Revealing Test

Fault Detection ประเมินว่าชุดทดสอบสามารถเปิดเผยพฤติกรรมผิดพลาดจริงได้หรือไม่ ในโครงการนี้ Candidate Test ที่ FAIL บน Buggy Version และ PASS บน Fixed Version จะถูกจัดเป็น Bug-Revealing Test และ Bug จะถูกนับว่า Detected เมื่อมี Bug-Revealing Test อย่างน้อยหนึ่งรายการ

2.14 ความสัมพันธ์ระหว่าง Coverage และ Fault Detection

Coverage และ Fault Detection เป็นตัวชี้วัดที่เกี่ยวข้องกันแต่ไม่เท่ากัน Test Suite อาจเข้าถึง Faulty Code แต่ไม่เปิดเผย Fault หาก Oracle/Assertion ไม่สังเกตความผิดปกติ งานของ Inozemtseva และ Holmes (2014) พบว่า Coverage ไม่ควรถูกใช้เป็นตัวแทน Test Suite Effectiveness เพียงอย่างเดียว

2.15 งานวิจัยที่เกี่ยวข้อง

Fraser และ Arcuri (2011) นำเสนอ EvoSuite สำหรับ Automatic Test Suite Generation ในโปรแกรมเชิงวัตถุ ขณะที่ Panichella et al. (2018) นำเสนอ DynaMOSA เพื่อจัดการ Many-Objective Coverage Targets แบบพลวัต

Abdullin et al. (2021) นำเสนอ KEX ใน SBST 2021 โดยอธิบาย Symbolic Execution และ Reanimator ซึ่งเป็น original backward-search based algorithm สำหรับสร้าง executable tests ส่วน Panichella et al. (2021) อธิบาย Reanimator ในรายงานการแข่งขันว่าเป็น novel backward search algorithm

Yang et al. (2024) ศึกษา LLM Unit Test Generation บน Java Projects และพบอิทธิพลของ Prompt Factors ส่วน Ouédraogo et al. (2024) ประเมิน LLM-Generated Tests ในขนาดใหญ่และพบทั้งศักยภาพและข้อจำกัดด้าน Compilability และ Test Quality โครงการนี้นำแนวคิดเหล่านี้มารวมใน Benchmark เดียวที่พิจารณา Evaluation, Coverage, Fault Detection และ Token Efficiency

บทที่ 3

วิธีดำเนินการและการออกแบบการทดลอง

3.1 ภาพรวมการดำเนินงาน

การทดลองเปรียบเทียบ DynaMOSA ผ่าน EvoSuite, Reanimator ผ่าน KEX, Gemini และ Claude โดยใช้ Defects4J เป็น Benchmark และกำหนดกระบวนการเดียวกันสำหรับการสร้าง

Test, Evaluation, Coverage และ Aggregationกระบวนการหลักประกอบด้วย การเตรียม Benchmark/Targets การสร้าง Test Case การ Compile/Evaluation กับ Buggy และ Fixed Versions การวัด Coverage และการรวบรวม Metrics เพื่อเปรียบเทียบผล

3.2 ชุดข้อมูลและ Benchmark ที่ใช้ในการทดลอง

Benchmark ประกอบด้วย 854 active bugs จาก 17 Java projects ใน Defects4J โดยใช้ 854 bugs เป็น denominator หลักสำหรับ Overall Evaluation Success, Coverage Success และ Fault Detection Rate

ตารางที่ 3.1 จำนวน Bugs ในแต่ละ Defects4J Project

ลำดับ	Project	จำนวน Bugs
1	Chart	26
2	Cli	39
3	Closure	174
4	Codec	18
5	Collections	28
6	Compress	47
7	Csv	16
8	Gson	18
9	JacksonCore	26
10	JacksonDatabind	110
11	JacksonXml	6
12	Jsoup	93
13	JXPath	22
14	Lang	61
15	Math	106
16	Mockito	38
17	Time	26
	รวม	854

3.3 การกำหนด Target Classes

สำหรับแต่ละ Bug ใช้ข้อมูล `classes.modified` เพื่อระบุ Target Classes เนื่องจากหนึ่ง Bug อาจแก้ไขหลาย Class จำนวน Target Entries จึงมากกว่าจำนวน Bugs ก่อนการรัน AI Benchmark มีการสร้าง Frozen Target Catalog เพื่อลดการเปลี่ยนแปลงของ Source Code ระหว่างการทดลอง

ตารางที่ 3.2 สรุปการคัดกรอง Target Entries สำหรับ Benchmark

รายการ	จำนวน
Dataset entries	1,073
SOURCE_NOT_FOUND	3
Non-Java resources	3
Java runnable targets	1,067

3.4 สภาพแวดล้อมในการทดลอง

โครงการใช้ Docker Environment กลางสำหรับ Defects4J, EvoSuite และ KEX โดย Repository ถูก mount ที่ `/workspace` ภายใน Container และกำหนด Java 11 เป็น Default Runtime ตาม Benchmark Protocol

ตารางที่ 3.3 สภาพแวดล้อมหลักของการทดลอง

รายการ	Configuration
Operating System ภายใน Container	Ubuntu 22.04
Container Platform	Docker / Docker Compose
Defects4J	3.0.1 ตาม Benchmark Protocol
Java หลัก	OpenJDK 11
Java Compatibility	OpenJDK 8 และ 17
Timezone	America/Los_Angeles
KEX	0.0.11
Python	Python 3
Build Tools	Maven และ Ant
Workspace	<code>/workspace</code>
KEX Working Data	<code>/root/kex-testing</code>

3.5 การสร้าง Test Case ด้วย DynaMOSA ผ่าน EvoSuite

Benchmark Runner เรียก Defects4J `gen_tests.pl` ด้วย generator `evosuite` และตั้งค่า - `Dalgorithm=DYNAMOSA` ใน `evosuite.config` อย่างชัดเจน เพื่อให้ DynaMOSA เป็น Search Algorithm ที่ใช้จริง

Target Classes ของ Bug ถูกเขียนเป็นรายการให้ `gen_tests.pl` และกำหนด Generation Budget 300 วินาทีต่อ Bug โดย Process ภายนอกมี Grace Period เพิ่ม 180 วินาทีสำหรับขั้นตอนที่เกี่ยวข้อง

Generation จะถือว่าสำเร็จเมื่อ Tool ทำงานสำเร็จและพบ Generated Test Files อย่างน้อยหนึ่งไฟล์

```
gen_tests.pl -g evosuite -p <PROJECT> -v <BUG_ID>b -n 1 -o
<OUTPUT> -b 300 -c <TARGET_CLASSES_FILE>
```

3.6 การสร้าง Test Case ด้วย Reanimator ผ่าน KEX

ในการทดลอง KEX ถูกใช้เป็นเครื่องมือสำหรับ Symbolic/Concolic Analysis และ Test Generation โดย Reanimator เป็นอัลกอริทึม backward search ที่ใช้ภายในกระบวนการสร้าง Test เพื่อสร้าง Object State และ executable code จาก symbolic inputs ตามแนวทางของ KEX (Abdullin et al., 2021)

Benchmark Runner เรียก KEX 0.0.11 ใน mode `concolic` แยกตาม Target Class และสร้าง Classpath จาก compiled project classes ร่วมกับ `cp.compile` ของ Defects4J

กำหนด Timeout 300 วินาทีต่อ Target Class หาก Bug มีหลาย Target จะจัดเก็บ Output แยก Directory เพื่อป้องกัน Artifact ทับกัน

```
python3 /opt/kex/kex.py --classpath
<PROJECT_CLASSES:CP_COMPILE> --target <TARGET_CLASS> --mode
concolic --output <OUTPUT_DIR>
```

3.7 การออกแบบ Prompt สำหรับ Gemini และ Claude

Prompt Template ถูกกำหนดล่วงหน้าและใช้ Source Code ของ Target เป็น Context โดย Input หลักประกอบด้วย Package Name, Class Name, Source Code และ Related Classes/Dependencies

ทั้งสอง Prompt กำหนด JUnit 4, Public Methods, Normal/Edge/Exception Cases, ห้าม Mocking Framework และให้ตอบเฉพาะ Java Code ที่สามารถ Compile ได้ โดย Claude ใช้ XML-like tags เพื่อจัดโครงสร้าง Prompt

Prompt ฉบับเต็มแสดงในภาคผนวก ก

3.8 Configuration ของ Gemini และ Claude

Parameter	Gemini	Claude
Model	gemini-3.7-flash	claude-sonnet-5
max_tokens	16,384	16,384
temperature	0.2	0.2
HTTP request timeout	300 วินาที	300 วินาที
Default delay	1.5 วินาที	1.5 วินาที

AI Runner รองรับ API Key Rotation, Progress Tracking, Resume และ Force Regeneration โดยบันทึก Prompt, Raw Result, TestCode, Usage, Finish Reason และ Elapsed Time ของแต่ละ Target

3.9 การจัดเก็บผลการสร้าง Test Case

ผลลัพธ์ถูกแยกตามเทคนิคเป็น DynaMOSA-EvoSuite/, Reanimator-Kex/, Gemini/ และ Claude/ โดย AI เก็บ Prompt, Result และ TestCode แยกกัน ทำให้สามารถย้อนกลับจาก Generated Test ไปยัง Prompt และ Raw API Response ได้

3.10 กระบวนการ Evaluation ของ Generated Tests

Evaluator Checkout และ Compile ทั้ง Buggy Version (<id>b) และ Fixed Version (<id>f) จากนั้นเตรียม Generated Sources ตามรูปแบบของแต่ละ Tool

Generated Tests ถูก Compile กับ Classpath ของ Buggy Version ก่อน หาก Compile ไม่ผ่านจะบันทึก evaluation_status = compile_failed

เมื่อ Compile สำเร็จ Candidate Test จะถูกรันกับ Buggy และ Fixed Version โดยกำหนด Timeout 15 วินาทีต่อ Candidate Test ต่อ Version สำหรับ EvoSuite/Gemini/Claude ใช้ระดับ JUnit Test Method ส่วน KEX ใช้ executable generated test classes และตัด Helper Classes ที่ไม่ใช่ Candidate Tests ออก

3.11 หลักเกณฑ์การจำแนกผลการทดสอบ

ตารางที่ 3.4 หลักเกณฑ์การจำแนก Candidate Test

Buggy	Fixed	Classification	ความหมาย
FAIL	PASS	bug_revealing	เปิดเผย Fault ที่ถูกแก้ไข
PASS	PASS	valid_non_revealing	Test ใช้งานได้แต่ไม่เปิดเผย Fault
FAIL	FAIL	invalid_or_unstable	ล้มเหลวทั้งสอง Version
PASS	FAIL	fixed_regression_or_unstable	พฤติกรรมไม่ตรงรูปแบบ Real-Fault Detection
TIMEOUT/อื่น ๆ	-	timeout_or_other	ไม่สามารถจำแนกในกรณีหลัก

Bug หนึ่งรายการถูกนับเป็น Detected Bug เมื่อมี bug_revealing อย่างน้อยหนึ่ง Candidate Test ดังนั้นจำนวน Bug-Revealing Tests อาจมากกว่าจำนวน Distinct Bugs Detected

3.12 การวัด Test Coverage

Generated Java Sources ถูกจัดตาม Package Structure และรวมเป็น .tar.bz2 ก่อนเรียก defects4j coverage -s <TEST_SUITE_ARCHIVE> บน Buggy Version

กำหนด Coverage Timeout 600 วินาทีต่อ Bug และสกัด Lines total/covered, Line coverage, Conditions total/covered และ Condition coverage จาก Output

กรณี Test Suite มี Failing Tests แต่ Defects4J ยังรายงาน Coverage Metrics ได้ จะยังถือ Coverage Status เป็น success เนื่องจาก Coverage และ Fault Detection เป็นคนละมิติ

3.13 ตัวชี้วัดที่ใช้ในการเปรียบเทียบ

- Evaluation Success Rate = $\text{Evaluation Success} / 854 \times 100$
- Overall FDR = $\text{Bugs Detected} / 854 \times 100$
- Conditional FDR = $\text{Bugs Detected} / \text{Evaluation Success} \times 100$
- Coverage Success Rate = $\text{Coverage Success} / 854 \times 100$
- Mean/Median Coverage คำนวณเฉพาะ Coverage Success และ Weighted Coverage คำนวณจากผลรวม Covered / Total ของ Cases ที่สำเร็จ
- Mean Condition Coverage พิจารณาเฉพาะ Cases ที่ conditions_total > 0

3.14 การเปรียบเทียบด้วย Common Case Set

โครงการสร้าง Common Evaluation Set จาก Intersection ของ Bugs ที่ทั้ง 4 เทคนิค Evaluation สำเร็จ และ Common Coverage Set จาก Intersection ของ Bugs ที่ทั้ง 4 เทคนิค Coverage สำเร็จ เพื่อเปรียบเทียบบน Cases เดียวกันและลด Selection Effect จาก Success Rate ที่แตกต่างกัน

3.15 การวิเคราะห์ Token ของ Gemini และ Claude

ใช้ Provider-recorded usage.total_tokens เป็นค่าหลัก โดยไม่สมมติว่า Total Tokens เท่ากับ Prompt + Completion เสมอไป ตัวชี้วัดประกอบด้วย Average/Median Tokens per Bug, Tokens per TestCode Bug, Tokens per Evaluation Success, Tokens per Detected Bug และ Detected Bugs per One Million Tokens

3.16 การจัดการ Failure, Timeout และ Missing Cases

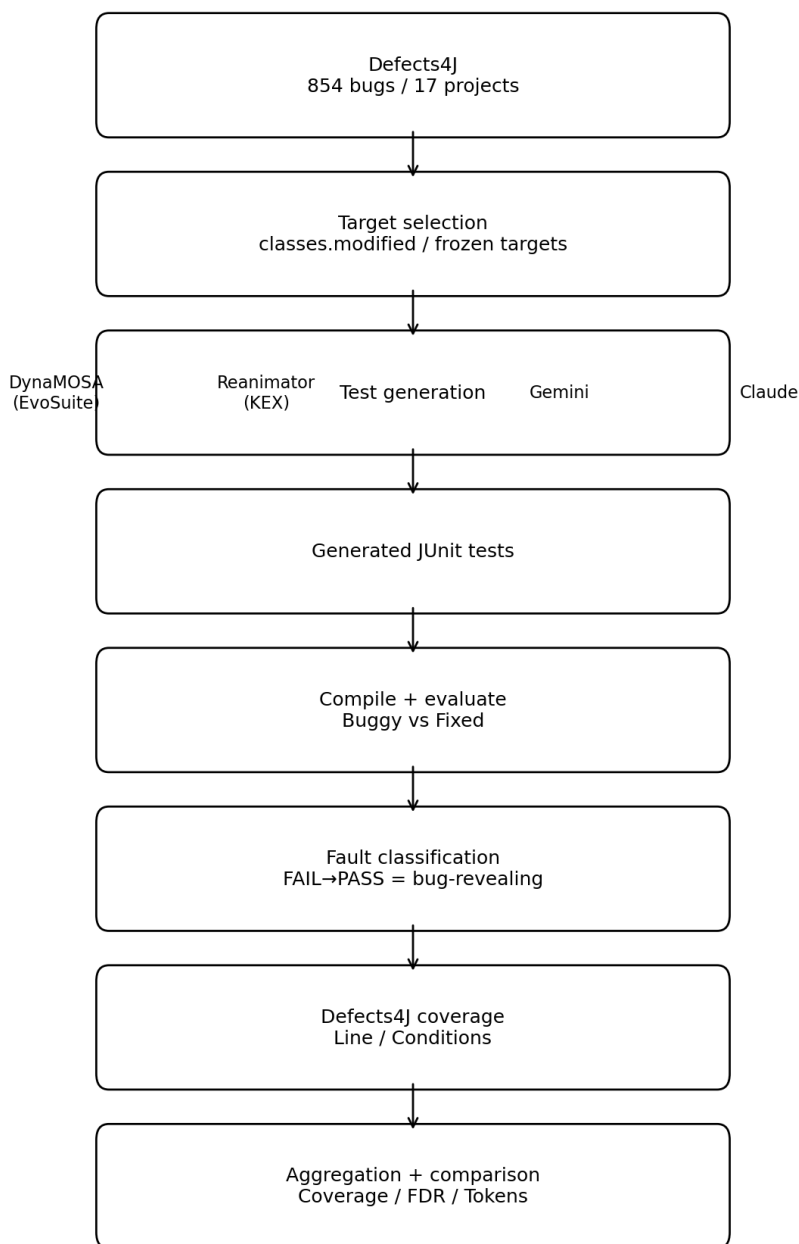
Generation, Evaluation และ Coverage มีสถานะแยกจากกัน เช่น success, failed, timeout, truncated, compile_failed, coverage_failed และ missing โดย Failed/Missing Cases ไม่ถูกแทนค่า

Coverage ด้วย 0% แต่ยังคงอยู่ใน denominator 854 เมื่อคำนวณ Overall Success/FDR ตาม Benchmark Protocol

3.17 การรวบรวมผลการทดลอง

Scripts สำหรับ Aggregation สร้าง master_results.csv, technique_summary.csv, status summaries, common-case summaries, project summary และ AI token summaries จาก Raw Evaluation/Coverage Results เพื่อป้องกันความคลาดเคลื่อนจากการคำนวณด้วยมือ

3.18 Workflow ของการทดลอง



รูปที่ 3.1 Workflow ของการทดลอง

Workflow แสดงลำดับตั้งแต่ Defects4J, Target Selection, Test Generation, Compilation/Evaluation, Fault Classification, Coverage และ Result Aggregation โดยใช้ขั้นตอนแยกจากกันเพื่อให้สามารถตรวจสอบ Failure ในแต่ละ Stage ได้

3.19 การออกแบบเพื่อให้สามารถทำซ้ำการทดลองได้

Repository เก็บ Source Code, Docker Configuration, Benchmark Scripts, Prompts, Generated Tests, Raw API Results, Evaluation Results, Coverage Results, Token Usage, Aggregation Scripts และ Final Summaries ส่วนขั้นตอนการรันซ้ำแสดงในภาคผนวก ข และ Configuration/Directory Structure แสดงในภาคผนวก ค

บทที่ 4

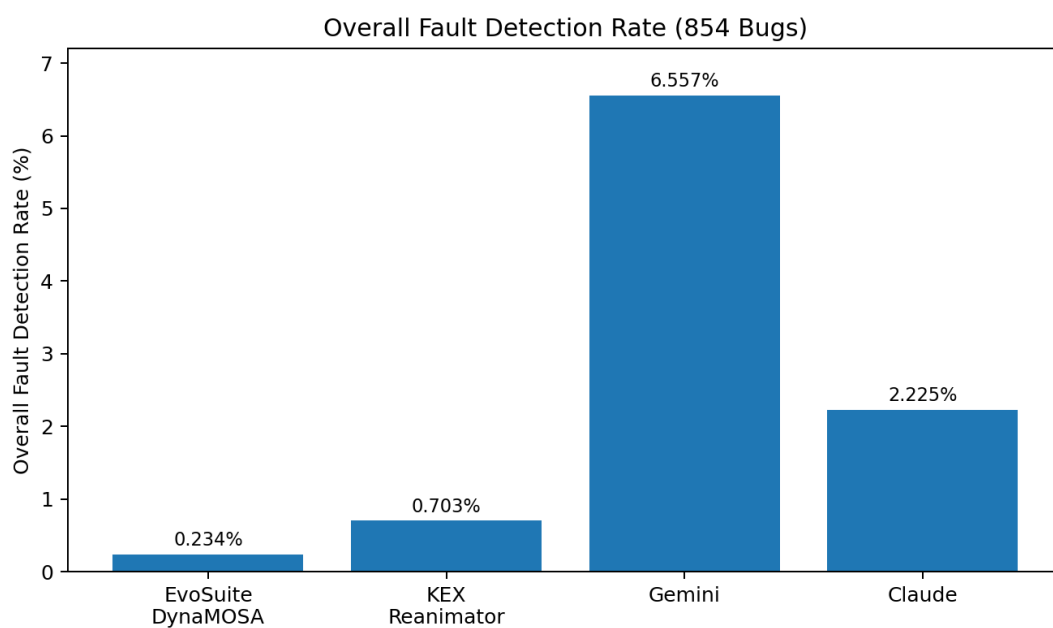
ผลการทดลอง

4.1 ภาพรวมผลการทดลอง

ผล Final Aggregation เปรียบเทียบทั้ง 4 เทคนิคบน Benchmark 854 bugs โดยพิจารณา Evaluation Success, Fault Detection และ Coverage Success

ตารางที่ 4.1 ผลการทดลองภาพรวมของทั้ง 4 เทคนิค

Technique	Eval Available	Eval Success	Eval / 854	Bugs Detected	Overall FDR	Coverage Success	Coverage / 854
DynaMOSA/EvoSuite	833	825	96.604%	2	0.234%	798	93.443%
Reanimator/KEX	684	684	80.094%	6	0.703%	507	59.368%
Gemini	843	485	56.792%	56	6.557%	471	55.152%
Claude	523	322	37.705%	19	2.225%	321	37.588%



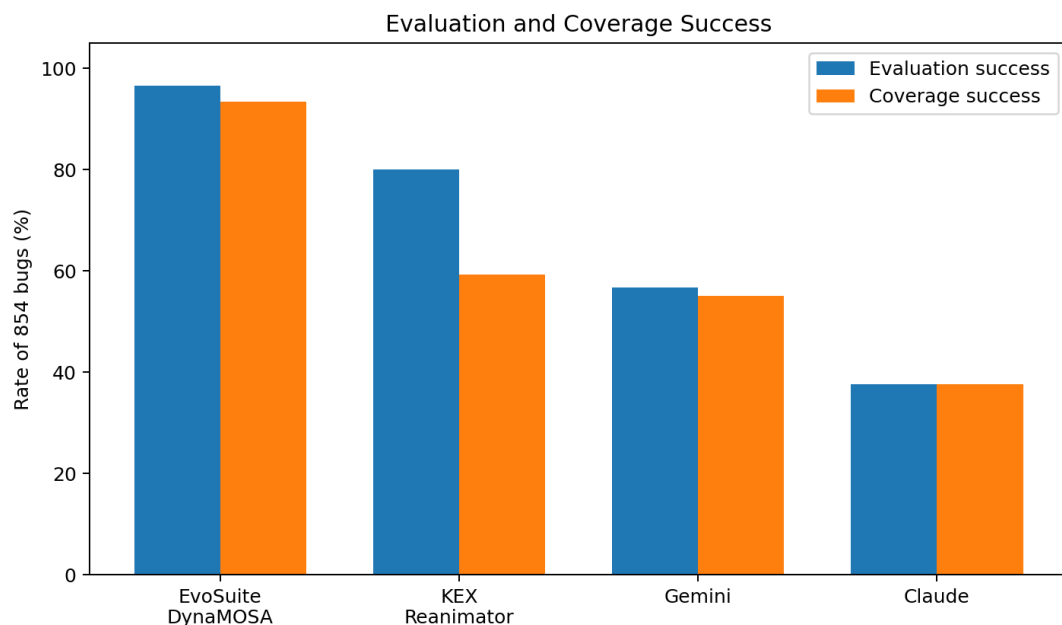
รูปที่ 4.1 Overall Fault Detection Rate ของทั้ง 4 เทคนิค

4.2 ผลด้าน Evaluation Success

ตารางที่ 4.2 สถานะ Evaluation ของแต่ละเทคนิค

Technique	Success	Compile Failed	Missing	รวม
DynaMOSA/EvoSuite	825	8	21	854
Reanimator/KEX	684	0	170	854
Gemini	485	358	11	854
Claude	322	201	331	854

DynaMOSA/EvoSuite มี Evaluation Success สูงที่สุด 825 bugs หรือ 96.604% รองลงมาคือ Reanimator/KEX 80.094% ขณะที่ Gemini และ Claude มี Compile Failure จำนวนมาก แสดงให้เห็นว่า AI Generation Success ไม่ได้เท่ากับ Executable Test Success



รูปที่ 4.2 เปรียบเทียบ Evaluation Success และ Coverage Success

4.3 ผลด้าน Fault Detection

ตารางที่ 4.3 Fault Detection ของทั้ง 4 เทคนิค

Technique	Eval Success	Bugs Detected	Overall FDR	FDR เมื่อ Eval สำเร็จ
DynaMOSA/EvoSuite	825	2	0.234%	0.242%
Reanimator/KEX	684	6	0.703%	0.877%
Gemini	485	56	6.557%	11.546%
Claude	322	19	2.225%	5.901%

Gemini มี Fault Detection สูงที่สุดทั้ง Overall FDR 6.557% และ Conditional FDR 11.546% รองลงมาคือ Claude ส่วน Reanimator/KEX และ DynaMOSA/EvoSuite ให้ FDR ต่ำกว่า แม้ Automatic Tools จะมี Evaluation Success สูงกว่า

4.4 Candidate Tests และ Bug-Revealing Tests

ตารางที่ 4.4 จำนวน Candidate Tests และ Bug-Revealing Tests

Technique	Candidate Tests	Bug-Revealing Tests	Bugs Detected
DynaMOSA/EvoSuite	46,457	2	2
Reanimator/KEX	65,099	22	6
Gemini	29,285	82	56
Claude	26,390	45	19

Reanimator/KEX มี Candidate Tests มากที่สุด 65,099 แต่ Gemini มี Bug-Revealing Tests และ Distinct Bugs Detected มากที่สุด แสดงว่าปริมาณ Candidate Tests ไม่สามารถใช้แทน Fault Detection Capability โดยตรง

4.5 ผลการวัด Test Coverage

ตารางที่ 4.5 ผล Test Coverage ของทั้ง 4 เทคนิค

Technique	Coverage Success	Mean Line	Median Line	Weighted Line	Mean Condition	Median Condition	Weighted Condition
DynaMOSA/EvoSuite	798	67.358%	80.25%	57.977%	62.862%	75.0%	52.446%
Reanimator/KEX	507	42.011%	42.8%	34.151%	30.501%	25.4%	24.564%
Gemini	471	85.799%	97.1%	81.081%	80.545%	92.3%	72.676%
Claude	321	79.626%	94.6%	64.836%	71.613%	85.05%	53.417%

Gemini มี Mean Line/Condition Coverage สูงที่สุดใน Cases ที่ Coverage สำเร็จ แต่ DynaMOSA/EvoSuite มี Coverage Success Rate สูงที่สุด ดังนั้นค่า Coverage เฉลี่ยควรพิจารณาควบคู่กับจำนวน Cases ที่สามารถวัดได้

4.6 สถานะของกระบวนการ Coverage

ตารางที่ 4.6 สถานะ Coverage ของทั้ง 4 เทคนิค

Technique	Success	Coverage Failed	Timeout	Missing	รวม
DynaMOSA/EvoSuite	798	35	0	21	854
Reanimator/KEX	507	177	0	170	854
Gemini	471	371	1	11	854
Claude	321	202	0	331	854

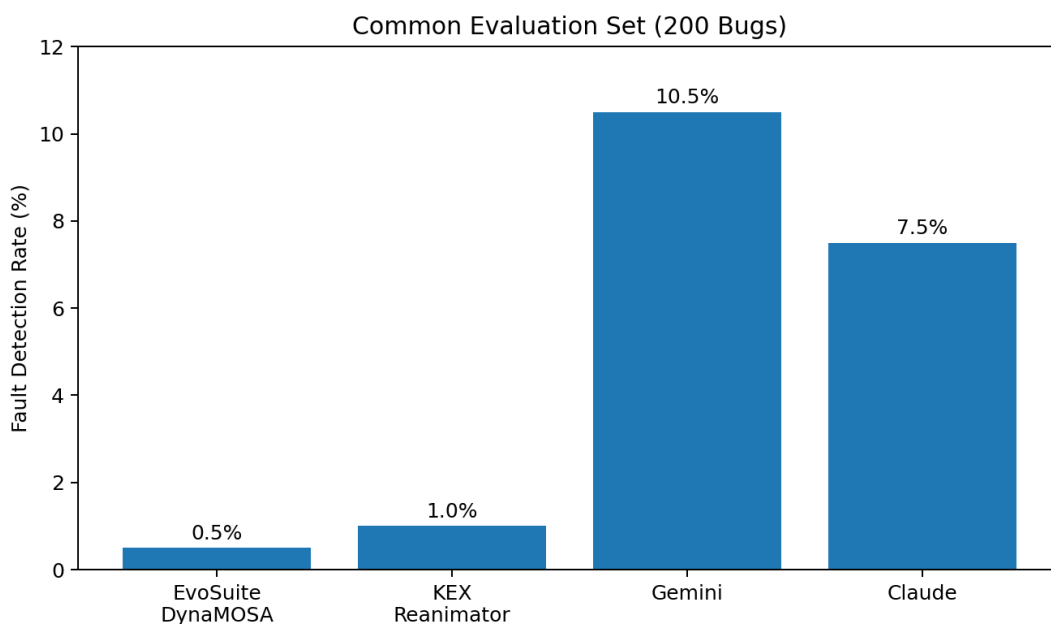
Gemini มี Timeout ที่เหลือใน Final Result 1 กรณีคือ Math-84 ซึ่งถูก rerun ด้วย Coverage Timeout เดิม 600 วินาทีแล้วและยัง Timeout จึงเก็บไว้เป็นผลตาม Protocol แทนการเพิ่ม Timeout เฉพาะ Case

4.7 การเปรียบเทียบบน Common Evaluation Set

ตารางที่ 4.7 Fault Detection บน Common Evaluation Set 200 Bugs

Technique	Common Cases	Bugs Detected	Common FDR	Bug-Revealing Tests
DynaMOSA/EvoSuite	200	1	0.5%	1
Reanimator/KEX	200	2	1.0%	12
Gemini	200	21	10.5%	33
Claude	200	15	7.5%	39

เมื่อควบคุมให้ทั้ง 4 เทคนิคถูกประเมินบน 200 bugs เดียวกัน Gemini ยังคงมี Common FDR สูงที่สุด 10.5% รองลงมาคือ Claude 7.5% แสดงให้เห็นว่าความแตกต่างด้าน Fault Detection ไม่ได้เกิดจาก Evaluation Set ที่ต่างกันเพียงอย่างเดียว



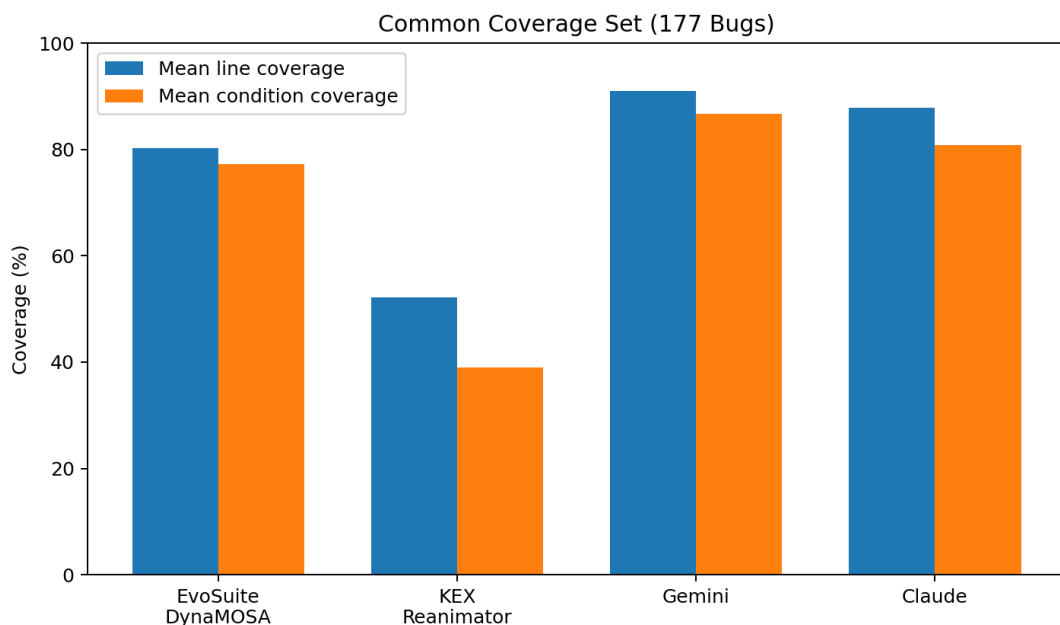
รูปที่ 4.3 Fault Detection Rate บน Common Evaluation Set

4.8 การเปรียบเทียบบน Common Coverage Set

ตารางที่ 4.8 Coverage บน Common Coverage Set 177 Bugs

Technique	Mean Line	Median Line	Weighted Line	Mean Condition	Median Condition	Weighted Condition
DynaMOSA/EvoSuite	80.272%	92.1%	80.265%	77.216%	91.95%	76.044%
Reanimator/KEX	52.126%	54.2%	44.981%	38.959%	35.7%	34.426%
Gemini	91.073%	98.9%	86.009%	86.648%	95.25%	78.364%
Claude	87.854%	96.7%	86.473%	80.814%	91.2%	75.878%

บน Common Coverage Set Gemini มี Mean Line 91.073% และ Mean Condition 86.648% สูงที่สุด ส่วน Claude มี Weighted Line 86.473% สูงกว่า Gemini 86.009% เล็กน้อย แสดงว่าการจัดอันดับอาจเปลี่ยนตามวิธี Aggregate Coverage



รูปที่ 4.4 Coverage บน Common Coverage Set

4.9 ผลการตรวจพบข้อบกพร่องแยกตาม Project

ตารางที่ 4.9 จำนวน Bugs Detected แยกตาม Project

Project	DynaMOSA	Reanimator	Gemini	Claude
Chart	0	0	7	1
Cli	0	0	2	0
Closure	1	2	4	0
Codec	0	0	5	1
Collections	0	0	5	2
Compress	0	0	4	2
Csv	0	0	0	0
Gson	0	0	2	0

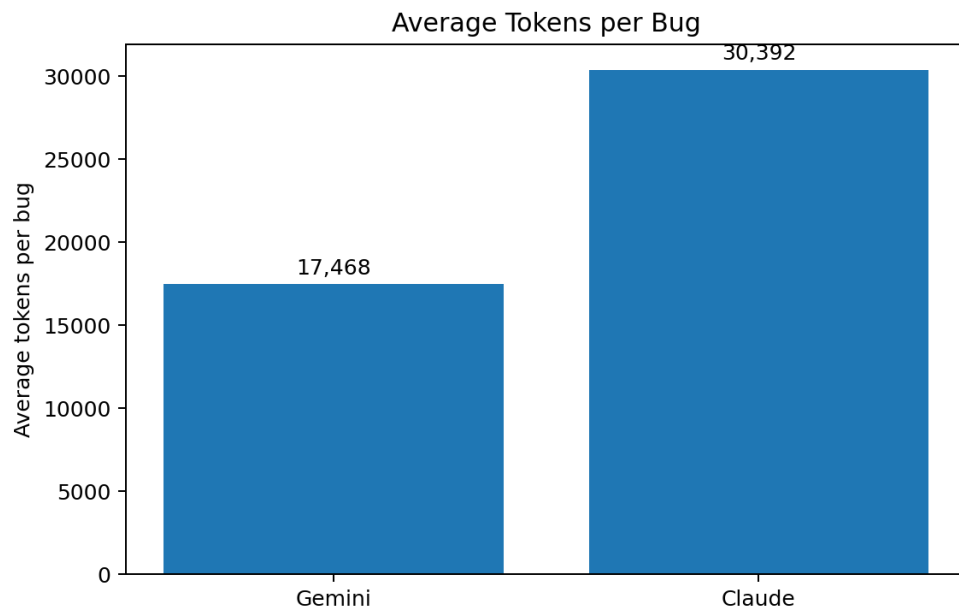
Project	DynaMOSA	Reanimator	Gemini	Claude
JacksonCore	0	0	0	1
JacksonDatabind	0	0	3	0
JacksonXml	0	0	0	0
Jsoup	0	2	3	2
JXPath	0	0	0	0
Lang	0	0	7	2
Math	0	2	10	6
Mockito	0	0	1	1
Time	1	0	3	1
รวม	2	6	56	19

Gemini ตรวจพบ Bugs ในหลาย Projects โดย Math สูงที่สุด 10 bugs ส่วน Reanimator/KEX ตรวจพบใน Closure, Jsoup และ Math อย่างละ 2 bugs และ DynaMOSA/EvoSuite ตรวจพบใน Closure และ Time อย่างละ 1 bug

4.10 ผลการวิเคราะห์ Token ของ Gemini และ Claude

ตารางที่ 4.10 การใช้ Token ของ Gemini และ Claude

Metric	Gemini	Claude
Bugs with Token Data	854/854	854/854
Unique API Calls	1,070	1,087
API Generation Success	1,050	600
API Generation Success Rate	98.131%	55.198%
Failed/Truncated Calls	20	487
Prompt Tokens	8,355,510	12,676,098
Completion Tokens	4,870,455	13,278,358
Total Tokens	14,918,091	25,954,456
Average Tokens / Bug	17,468.49	30,391.63
Median Tokens / Bug	12,745.00	23,691.00



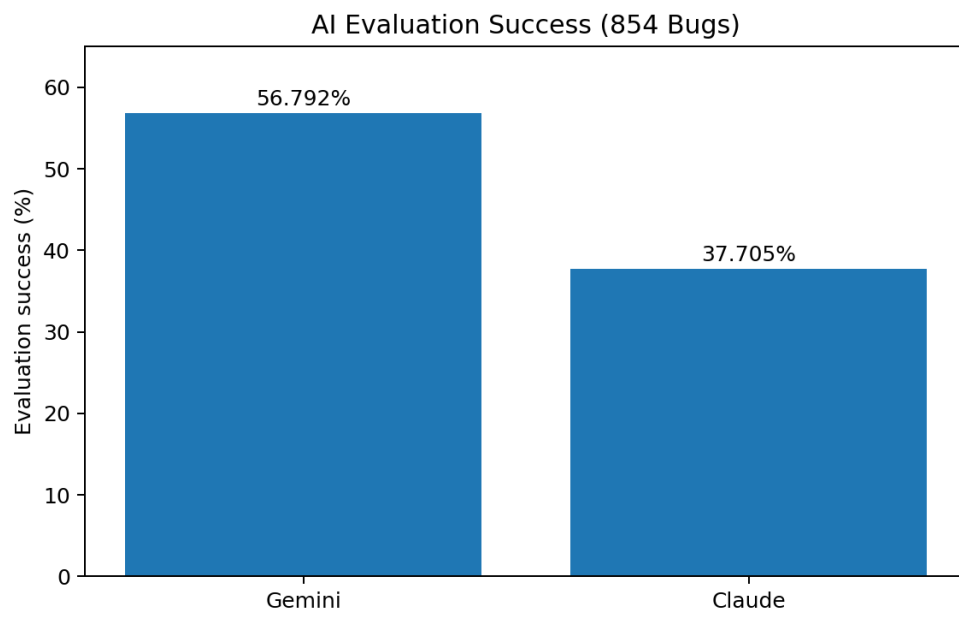
รูปที่ 4.5 Average Tokens per Bug

4.11 ประสิทธิภาพของ AI เมื่อเทียบกับ Token ที่ใช้

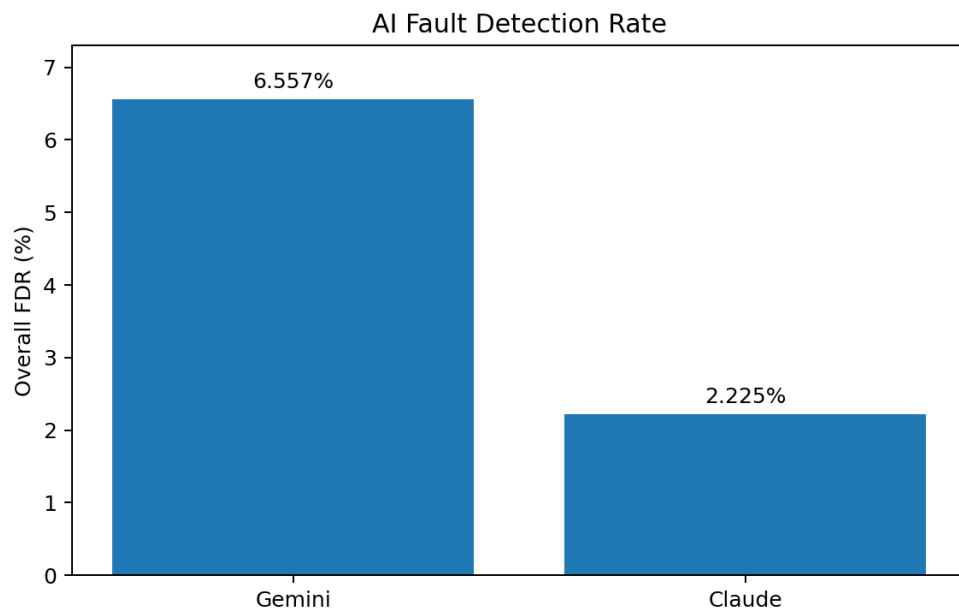
ตารางที่ 4.11 Token Efficiency ของ Gemini และ Claude

Metric	Gemini	Claude
TestCode Available	843/854	523/854
Evaluation Success	485/854	322/854
Bugs Detected	56/854	19/854
Tokens / TestCode Bug	17,696.43	49,626.11
Tokens / Evaluation Success	30,758.95	80,603.90
Tokens / Detected Bug	266,394.48	1,366,024.00
Detected Bugs / 1M Tokens	3.7538	0.7321

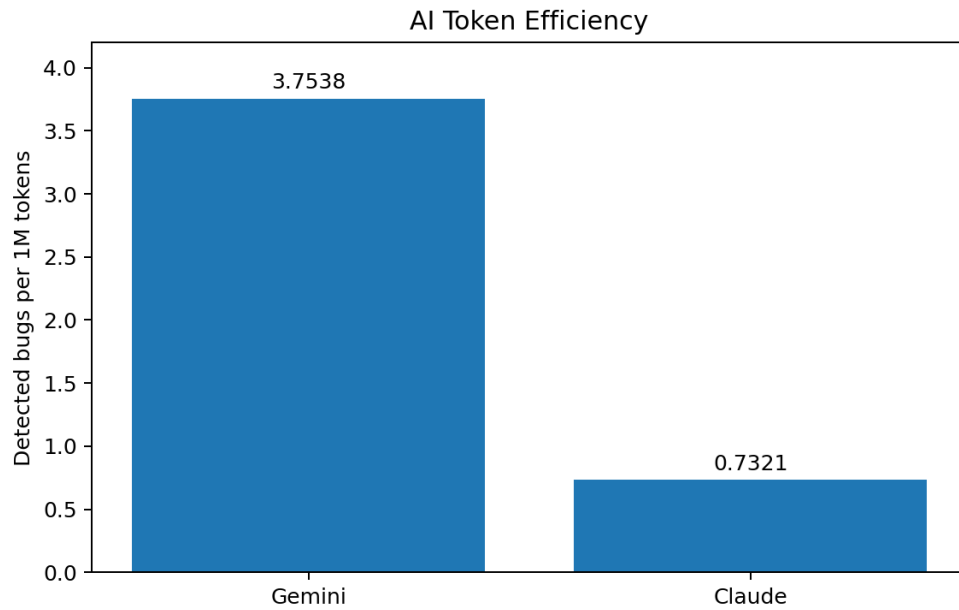
Gemini ใช้ Total Tokens น้อยกว่า Claude พร้อมกับมี TestCode Availability, Evaluation Success และ Fault Detection สูงกว่าใน Benchmark นี้ ทำให้ Detected Bugs per One Million Tokens สูงกว่าประมาณ 5.13 เท่า อย่างไรก็ตามผลดังกล่าวควรตีความเฉพาะ Configuration และ Model Versions ที่ใช้ในการทดลอง



รูปที่ 4.6 AI Evaluation Success



รูปที่ 4.7 AI Fault Detection Rate



รูปที่ 4.8 Detected Bugs per One Million Tokens

4.12 สรุปผลการทดลองเชิงเปรียบเทียบ

ตารางที่ 4.12 สรุปผลเด่นของเทคนิคทั้ง 4

ด้านที่พิจารณา	ผลที่เด่นที่สุด
Evaluation Success Rate	DynaMOSA/EvoSuite — 96.604%
Coverage Success Rate	DynaMOSA/EvoSuite — 93.443%
Candidate Tests	Reanimator/KEX — 65,099
Bug-Revealing Tests	Gemini — 82
Bugs Detected	Gemini — 56
Overall FDR	Gemini — 6.557%
Mean Line Coverage	Gemini — 85.799%
Mean Condition Coverage	Gemini — 80.545%
Common Evaluation FDR	Gemini — 10.5%
AI Detected Bugs / 1M Tokens	Gemini — 3.7538

ผลโดยรวมแสดงว่าไม่มีเทคนิคใดดีที่สุดในทุกมิติ DynaMOSA/EvoSuite เด่นด้าน Reliability, Reanimator/KEX มี Candidate Tests และ Evaluation Success สูง, Gemini เด่นด้าน Fault Detection และ Coverage ใน Success Cases ส่วน Claude มี Fault Detection สูงกว่า Automatic Tools บางเทคนิคแต่ Success Rate ต่ำกว่า Gemini

บทที่ 5

สรุปผล อภิปรายผล และข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

โครงการเปรียบเทียบ DynaMOSA ผ่าน EvoSuite, Reanimator ผ่าน KEX, Gemini และ Claude บน Defects4J 854 bugs โดยใช้ Evaluation, Coverage, Fault Detection และ Token Efficiency เป็นตัวชี้วัดหลัก

DynaMOSA/EvoSuite มี Evaluation และ Coverage Success สูงที่สุด ขณะที่ Gemini ตรวจพบ Bugs มากที่สุดและมี Mean Coverage สูงใน Cases ที่สำเร็จ Reanimator/KEX มี Evaluation Success สูงเป็นอันดับสองและ Candidate Tests มากที่สุด ส่วน Claude มีผลอยู่ระหว่าง Automatic Tools กับ Gemini ในหลายตัวชี้วัด

5.2 อภิปรายผลการทดลอง

5.2.1 ความแตกต่างระหว่างความเสถียรและ Fault Detection

DynaMOSA/EvoSuite มี Operational Reliability สูงที่สุด แต่ Fault Detection ต่ำกว่ากลุ่ม AI อย่างมาก ขณะที่ Gemini มี Success Rate ต่ำกว่าแต่ตรวจพบ Fault มากกว่า ผลนี้สนับสนุนแนวคิดว่า Coverage หรือ Executability ไม่ควรถูกใช้แทน Test Effectiveness เพียงอย่างเดียว ซึ่งสอดคล้องกับ Inozemtseva และ Holmes (2014)

5.2.2 ผลของ DynaMOSA ผ่าน EvoSuite

DynaMOSA ออกแบบเพื่อ Optimize Coverage Objectives จึงสามารถสร้าง Test Suites ที่ทำงานได้สม่ำเสมอ อย่างไรก็ตามการเข้าถึง Code Path ไม่ได้รับประกันว่า Assertions จะเปิดเผย Real Fault จึงอธิบายได้ว่าทำไม Coverage Success สูงแต่ FDR ต่ำใน Benchmark นี้

5.2.3 ผลของ Reanimator ผ่าน KEX

Reanimator เป็น backward-search algorithm ที่ทำงานร่วมกับ symbolic inputs ของ KEX เพื่อสร้าง executable object construction code (Abdullin et al., 2021) ผลการทดลองมี Evaluation Success สูงและ Candidate Tests จำนวนมาก แต่ Coverage และ Fault Detection ต่ำกว่าเทคนิคอื่นบางกลุ่ม ข้อจำกัดดังกล่าวอาจเกี่ยวข้องกับ Symbolic/Concolic Analysis, Constraints, Object State Reconstruction และ Project Compatibility แต่ผลของโครงการไม่เพียงพอที่จะชี้สาเหตุใดสาเหตุหนึ่งโดยตรง

5.2.4 ผลของ Gemini

Gemini มี TestCode Availability สูง 843/854 แต่ Compile Failed 358 bugs แสดงช่องว่างระหว่าง Code Generation กับ Executable Test Generation อย่างชัดเจน อย่างไรก็ตามเมื่อ Evaluation สำเร็จ Gemini มี Conditional FDR 11.546% และให้ Coverage สูง ผลดังกล่าวสอดคล้องกับงาน LLM Test Generation ที่ชี้ทั้งศักยภาพและปัญหาด้าน Compilability/Hallucination (Yang et al., 2024; Ouédraogo et al., 2024)

5.2.5 ผลของ Claude

Claude มี TestCode Availability และ Evaluation Success ต่ำกว่า Gemini แต่บน Common Evaluation Set สร้าง Bug-Revealing Tests 39 tests มากกว่า Gemini 33 tests แม้ตรวจพบ Unique Bugs น้อยกว่า แสดงให้เห็นว่าจำนวน Bug-Revealing Tests และ Distinct Bugs Detected เป็นคนละมิติ

5.2.6 Common Case Analysis

Common Evaluation 200 bugs และ Common Coverage 177 bugs ช่วยลดอิทธิพลจาก Success Sets ที่แตกต่างกัน ผลยังคงแสดงว่า Gemini มี Fault Detection และ Mean Coverage สูงใน Same-Case Comparison จึงเพิ่มความเชื่อมั่นว่าผลไม่ได้เกิดจาก Selection Effect เพียงอย่างเดียว แต่ Overall Analysis ยังจำเป็นเพราะ Failure เองเป็นส่วนหนึ่งของประสิทธิภาพระบบ

5.3 การอภิปรายผลด้าน AI Token Efficiency

Gemini ใช้ 14,918,091 Total Tokens ขณะที่ Claude ใช้ 25,954,456 tokens และมี Detected Bugs per One Million Tokens เท่ากับ 3.7538 เทียบกับ 0.7321 ผลดังกล่าวแสดงว่า Gemini มี Token Efficiency สูงกว่าใน Benchmark นี้ แต่ไม่ควรถูกสรุปเป็นคุณสมบัติทั่วไปนอก Prompt, Model Version และ Provider Configuration ที่ใช้

5.4 ผลลัพธ์การเรียนรู้ที่ได้รับจากการทดลอง

- Generation Success ไม่เท่ากับ Compilation/Evaluation/Coverage Success จึงต้องมี Executable Validation Pipeline
- Coverage และ Fault Detection เป็นคนละมิติ และ Test Oracle มีผลต่อการเปิดเผย Fault
- Generative AI มีศักยภาพในการสร้าง Tests แต่จำเป็นต้องตรวจสอบ API Hallucination, Compile Error และ Dependency Compatibility
- Experimental Protocol, Fixed Denominator และ Common Case Analysis มีความสำคัญต่อความยุติธรรมของ Benchmark
- Reproducibility ต้องเก็บ Intermediate Artifacts ไม่ใช่เพียง Final Summary

- Progress Tracking ของ Long-Running Benchmark ควรตรวจสอบกับ Artifacts จริง ไม่ควรเชื่อ Progress File เพียงแหล่งเดียว

5.5 ปัญหาที่พบในการดำเนินงาน

- Build Environment และ Dependencies แตกต่างกันระหว่าง 17 Projects
- AI-Generated Tests มี Compile Failure จำนวนมาก โดย Gemini 358 bugs และ Claude 201 bugs
- Coverage Pipeline มี coverage_failed จำนวนมากในบางเทคนิค แม้ Generation/Evaluation จะสำเร็จ
- มี Timeout ใน Long-Running Cases เช่น Gemini Math-84 ซึ่งยัง Timeout เมื่อ rerun ภายใต้อายุ 600 วินาที
- เคยพบ Artifact ของ Gemini Csv-16 เสียหาย จึงต้อง regenerate ด้วย Model เดิมก่อนวัด Coverage ใหม่
- AI Benchmark ต้องจัดการ Quota ด้วย API Key Rotation, Resume และ Progress Tracking
- Output Structure ของ EvoSuite, KEX และ AI แตกต่างกัน จึงต้องมี Preparation Logic แยกตาม Tool

5.6 ข้อจำกัดของการทดลอง

- จำกัดเฉพาะ Java และ Defects4J จึงไม่สามารถสรุปไปยังภาษา/ระบบอื่นโดยตรง
- ผล AI จำกัดตาม Model Versions gemini-3.7-flash และ claude-sonnet-5 รวมถึง Prompt Templates ที่ใช้
- EvoSuite Benchmark ไม่ได้กำหนด Fixed Random Seed เพิ่มเติม จึงอาจมีความแปรปรวนเมื่อรันซ้ำ
- หลายส่วนของ Benchmark ใช้ Single Run เนื่องจากต้นทุนเวลาและ Token
- Candidate Test Unit ของ KEX แตกต่างจาก EvoSuite/Gemini/Claude จึงไม่ควรเปรียบเทียบ Candidate Count เป็นหน่วยที่เท่ากันสมบูรณ์
- ไม่ได้ทำ Statistical Hypothesis Testing เพื่อสรุป Statistical Significance ระหว่างทุกเทคนิค

5.7 Threats to Validity

5.7.1 Internal Validity

ปัจจัยภายในประกอบด้วย Runtime/Dependency Compatibility, Timeout, API Variability และ Randomness ของ Search-Based Algorithm โครงงานลดผลด้วย Docker Environment, Protocol กลาง และการบันทึก Configuration แต่ไม่สามารถกำจัดความแปรปรวนทั้งหมดได้

5.7.2 Construct Validity

โครงงานใช้หลาย Metrics เพื่อไม่ให้ Coverage เป็นตัวแทน Test Effectiveness เพียงค่าเดียว อย่างไรก็ตามนิยาม Bug-Revealing Test แบบ Buggy FAIL → Fixed PASS เป็น Operational Definition ที่อาจไม่ครอบคลุม Fault Detection ทุกรูปแบบ

5.7.3 External Validity

ผลจาก 17 Java Projects ใน Defects4J อาจไม่ทั่วไปต่อระบบ GUI, Database, Network, Concurrent หรือภาษาอื่น ดังนั้นการนำผลไปใช้นอกบริบทควรทำด้วยความระมัดระวัง

5.7.4 Conclusion Validity

ผลรายงานเป็น Descriptive Metrics และ Same-Case Analysis แต่ไม่มี Statistical Hypothesis Testing จึงควรสรุปว่าเทคนิคหนึ่งให้ค่าที่สูงกว่าอีกเทคนิคภายใต้ Benchmark/Configuration นี้ ไม่ใช่ข้อสรุปทั่วไปต่อประชากรซอฟต์แวร์ทั้งหมด

5.8 ข้อเสนอแนะและแนวทางการพัฒนาต่อ

- ทดลอง Hybrid Pipeline ที่ให้ Automatic Tool สร้าง Baseline Tests แล้วใช้ LLM สร้าง Targeted Tests สำหรับ Uncovered/Weak Areas
- ใช้ Compile Error และ Coverage Report เป็น Feedback สำหรับ LLM Repair แต่ต้องแยก Baseline กับ Repair-Assisted Results
- รัน EvoSuite หลาย Random Seeds และรัน AI หลาย Independent Generations เพื่อวัด Variance
- จำแนก Compile Failures ของ AI เป็น Wrong API, Wrong Signature, Missing Dependency, Access Modifier, Syntax Error และประเภทอื่น
- ศึกษาความสัมพันธ์ระหว่าง Source Complexity กับ Generation Success/Coverage/Fault Detection

- เพิ่ม Monetary Cost, Wall-Clock Time และ API Latency ในการวิเคราะห์ Cost-Effectiveness
- ขยาย Benchmark ไปยังภาษาและระบบประเภทอื่น

5.9 สรุปผลโครงการ

ผลการทดลองแสดงให้เห็นว่า Automatic Test Generation Algorithms และ Generative AI มีจุดแข็งต่างกัน DynaMOSA/EvoSuite เด่นด้าน Reliability, Reanimator/KEX ให้ Evaluation Success สูง และผลิต Candidate Tests จำนวนมาก, Gemini เด่นด้าน Fault Detection และ Coverage ใน Success Cases ส่วน Claude มี Fault Detection สูงกว่า Automatic Tools บางเทคนิคใน Benchmark นี้ จึงยังไม่ควรสรุปว่าแนวทางหนึ่งแทนอีกแนวทางได้ทั้งหมด แต่มีศักยภาพในการนำจุดแข็งของทั้งสองกลุ่มมาผสมผสานกัน

รายการอ้างอิง

- Abdullin, A., Akhin, M., & Belyaev, M. (2021). Kex at the 2021 SBST tool competition. In 2021 IEEE/ACM 14th International Workshop on Search-Based Software Testing (SBST) (pp. 32–33). IEEE. <https://doi.org/10.1109/SBST52555.2021.00014>
- Anthropic. (n.d.). Claude API documentation. Retrieved October 4, 2026, from <https://docs.anthropic.com/>
- Defects4J. (n.d.). Defects4J: A database of existing faults to enable controlled testing studies for Java programs [Computer software]. GitHub. Retrieved October 4, 2026, from <https://github.com/rjust/defects4j>
- Fraser, G., & Arcuri, A. (2011). EvoSuite: Automatic test suite generation for object-oriented software. In Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (pp. 416–419). Association for Computing Machinery. <https://doi.org/10.1145/2025113.2025179>
- Google. (n.d.). Gemini API documentation. Google AI for Developers. Retrieved October 4, 2026, from <https://ai.google.dev/gemini-api/docs>
- Harman, M., Mansouri, S. A., & Zhang, Y. (2012). Search-based software engineering: Trends, techniques and applications. *ACM Computing Surveys*, 45(1), Article 11, 1–61. <https://doi.org/10.1145/2379776.2379787>
- Inozemtseva, L., & Holmes, R. (2014). Coverage is not strongly correlated with test suite effectiveness. In Proceedings of the 36th International Conference on Software Engineering (pp. 435–445). Association for Computing Machinery. <https://doi.org/10.1145/2568225.2568271>
- International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2022). ISO/IEC/IEEE 29119-1:2022 Software and systems engineering—Software testing—Part 1: General concepts. International Organization for Standardization.
- Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In Proceedings of the 2014 International Symposium on Software Testing and Analysis (pp. 437–440). Association for Computing Machinery. <https://doi.org/10.1145/2610384.2628055>
- King, J. C. (1976). Symbolic execution and program testing. *Communications of the ACM*, 19(7), 385–394. <https://doi.org/10.1145/360248.360252>

- Ouédraogo, W. C., Kaboré, A. K., Tian, H., Song, Y., Koyuncu, A., Klein, J., Lo, D., & Bissyandé, T. F. (2024). Large-scale, independent and comprehensive study of the power of LLMs for test case generation. CoRR, abs/2407.00225. <https://doi.org/10.48550/arXiv.2407.00225>
- Panichella, A., Kifetew, F. M., & Tonella, P. (2018). Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets. *IEEE Transactions on Software Engineering*, 44(2), 122–158. <https://doi.org/10.1109/TSE.2017.2663435>
- Panichella, S., Gambi, A., Zampetti, F., & Riccio, V. (2021). SBST tool competition 2021. In 2021 IEEE/ACM 14th International Workshop on Search-Based Software Testing (SBST) (pp. 20–27). IEEE. <https://doi.org/10.1109/SBST52555.2021.00011>
- Vorpai Research. (n.d.). Kex [Computer software]. GitHub. Retrieved October 4, 2026, from <https://github.com/vorpai-research/kex>
- Yang, L., Yang, C., Gao, S., Wang, W., Wang, B., Zhu, Q., Chu, X., Zhou, J., Liang, G., Wang, Q., & Chen, J. (2024). On the evaluation of large language models in unit test generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (pp. 1607–1619). Association for Computing Machinery. <https://doi.org/10.1145/3691620.3695529>

ภาคผนวก ก

Prompt Templates ที่ใช้ในการทดลอง

ก.1 Gemini Prompt Template

คุณคือ Java testing expert คุณต้องเขียน JUnit 4 test suite สำหรับคลาสที่เข้ามา โดยมีเป้าหมายเพื่อให้ได้ code coverage (line/branch) สูงที่สุดเท่าที่เป็นไปได้

ข้อมูล Class ภายใต้การทดสอบ

```
Package: {package_name}
Class name: {class_name}
Full source code:
```java
{source_code}
```
```

Dependencies ที่เกี่ยวข้อง: {related_classes}

ข้อกำหนดของ Test Suite

1. เขียนเป็น JUnit 4 เท่านั้น (import org.junit.Test, org.junit.Assert)
2. ครอบคลุม public method ทุกตัวของคลาสอย่างน้อย 1 test case
3. ต้องมี test case ทั้งกรณี normal input, edge case (เช่น null, 0, ค่าติดลบ, string ว่าง, array ว่าง), และกรณีที่ควร throw exception (ถ้ามี)
4. ห้ามใช้ mocking framework ใด ๆ ให้ทดสอบผ่าน public API จริงเท่านั้น
5. แต่ละ test method ต้องมีชื่อสื่อความหมาย (เช่น testMethodName_condition_expectedResult)
6. ห้ามเขียนคำอธิบายยาว ๆ ให้ตอบกลับเป็นโค้ด Java ที่คอมไพล์ได้จริงเท่านั้น ไม่ต้องมี markdown fence หรือคำอธิบายอื่น

ส่งออกเฉพาะโค้ด Java ของไฟล์ test class ชื่อ {class_name}Test.java เท่านั้น

ก.2 Claude Prompt Template

คุณคือ Java testing expert ที่เชี่ยวชาญด้าน unit testing และ JUnit framework งานของคุณคือเขียน JUnit 4 test suite ให้กับคลาสที่อยู่ใน tag <source_code> ด้านล่าง เป้าหมายหลักคือให้ได้ line coverage และ branch coverage สูงที่สุดเท่าที่เป็นไปได้

```
<class_info>
Package: {package_name}
Class name: {class_name}
</class_info>
```

```
<source_code>
{source_code}
</source_code>
```

```
<dependencies>
{related_classes}
</dependencies>
```

```
<requirements>
1. เขียนเป็น JUnit 4 เท่านั้น (import org.junit.Test, org.junit.Assert,
org.junit.Before ถ้าจำเป็น)
2. ครอบคลุม public method ทุกตัวอย่างน้อย 1 test case ต่อ method
3. ต้องมี test case ครบทั้ง 3 กลุ่ม: (ก) normal/typical input (ข) edge case
เช่น null, 0, ค่าติดลบ, string ว่าง, boundary value (ค) กรณีที่ควร throw
exception พร้อมใช้ @Test(expected=...) หรือ try-catch ตรวจสอบ
4. ห้ามใช้ mocking framework ใด ๆ ทดสอบผ่าน public API ของคลาสจริงเท่านั้น
5. ตั้งชื่อ test method ให้สื่อความหมายตามรูปแบบ
testMethodName_condition_expectedResult
6. ห้าม import class ที่ไม่มีอยู่จริงหรือเอา API ที่ไม่ได้ให้มา ถ้าจำเป็นต้องใช้ dependency
ที่ไม่มีข้อมูลเพียงพอ ให้ระบุไว้เป็น comment ในโค้ดแทนการเอา
</requirements>
```

```
<output_format>
ตอบกลับเฉพาะโค้ด Java ที่คอมไพล์ได้จริงเท่านั้น ห้ามมีคำอธิบายก่อน/หลังโค้ด ห้ามใช้ markdown code fence
ชื่อไฟล์/คลาสคือ {class_name}Test.java
</output_format>
```

ภาคผนวก ข

Reproducibility Guide

ภาคผนวกนี้สรุปขั้นตอนหลักสำหรับทำซ้ำการทดลองจาก Repository โดยไม่เปิดเผย API Key จริง ผู้ทำซ้ำต้องใช้ Environment และ Credentials ที่ตนมีสิทธิ์ใช้งาน

ข.1 สร้างและเปิด Docker Environment

```
cd docker
docker compose up -d --build
docker exec -it sqa_kex_container bash
cd /workspace
```

ข.2 รัน Automatic Test Generation

```
python3 scripts/run_benchmark.py --all-bugs --tool evosuite
--resume
python3 scripts/run_benchmark.py --all-bugs --tool kex --
resume
```

ข.3 รัน AI Test Generation

ตั้งค่า KKU_API_KEY หรือชุด Key ตามที่ AI Runner รองรับ จากนั้นรัน Full Frozen Dataset โดยไม่ใช้ --force เพื่อให้ระบบ Resume จาก Artifacts ที่เสร็จแล้ว

```
export KKU_API_KEY="<YOUR_AUTHORIZED_KEY>"
python3 scripts/ai_benchmark_runner.py --model all --max-
tokens 16384 --temperature 0.2
```

สามารถตรวจสอบสถานะโดยใช้ python3 scripts/ai_benchmark_runner.py --status และใช้ --dry-run เพื่อตรวจสอบรายการ Target ก่อนยิง API จริง

ข.4 รัน Evaluation

```
python3 scripts/batch_evaluate.py --tool evosuite
python3 scripts/batch_evaluate.py --tool kex
python3 scripts/batch_evaluate.py --tool gemini
python3 scripts/batch_evaluate.py --tool claude
```

Evaluator ใช้ timeout 15 วินาทีต่อ Candidate Test ต่อ Version เป็นค่าเริ่มต้น หากต้องการตรวจสอบเฉพาะ Bug สามารถใช้ --project และ --bug

ข.5 รัน Coverage

```
python3 scripts/run_coverage_benchmark.py --tool evosuite --
timeout 600
python3 scripts/run_coverage_benchmark.py --tool kex --
timeout 600
```

```
python3 scripts/run_coverage_benchmark.py --tool gemini --
timeout 600
python3 scripts/run_coverage_benchmark.py --tool claude --
timeout 600
```

ข.6 Aggregate และสร้างกราฟ

```
python3 scripts/aggregate_results.py
python3 scripts/aggregate_ai_tokens.py
python3 scripts/generate_final_charts.py
python3 scripts/generate_ai_token_charts.py
```

ข.7 ตรวจสอบผลลัพธ์

- ตรวจสอบ evaluation/final/master_results.csv ว่ามี 3,416 data rows = 854 bugs x 4 techniques
- ตรวจสอบ technique_summary.csv, common_evaluation_summary.csv, common_coverage_summary.csv และ ai_token_summary.csv
- ตรวจสอบ Generated TestCode, Prompt/Raw Result และ summary.json ของ Case ที่ผลิตปกติก่อนสรุปผล
- ไม่แก้ไข Generated Tests ด้วยมือเพื่อบังคับให้ผ่าน Evaluation/Coverage เพราะจะทำให้ Benchmark ไม่สะท้อนผลของ Tool/Model

ภาคผนวก ค

โครงสร้างผลลัพธ์และ Configuration

ค.1 โครงสร้าง Repository ที่เกี่ยวข้อง

```

ProjectSQA/
├── DynaMOSA-EvoSuite/
│   ├── TestCode/
│   └── Result_Round2/
├── Reanimator-Kex/
│   ├── TestCode/
│   └── Result_Round2/
├── Gemini/
│   ├── Prompt/
│   ├── Result/
│   └── TestCode/
├── Claude/
│   ├── Prompt/
│   ├── Result/
│   └── TestCode/
├── evaluation/
│   ├── evosuite/
│   ├── kex/
│   ├── gemini/
│   ├── claude/
│   ├── coverage/
│   └── final/
├── scripts/
└── docker/
  
```

ค.2 Configuration หลัก

ส่วน	Configuration
Benchmark	Defects4J 3.0.1; 854 active bugs / 17 projects
Target catalog	1,073 entries → 1,067 Java runnable targets
DynaMOSA/EvoSuite	algorithm=DYNAMOSA; generation budget 300s/bug; process grace 180s
Reanimator/KEX	KEX 0.0.11; mode=concolic; timeout 300s/target class
Gemini	gemini-3.7-flash; max_tokens=16384; temperature=0.2; request timeout=300s
Claude	claude-sonnet-5; max_tokens=16384; temperature=0.2; request timeout=300s
Evaluation	Buggy/Fixed execution; timeout 15s per candidate per version
Coverage	defects4j coverage -s; timeout 600s/bug
Coverage metrics	Line Coverage และ Condition Coverage ตาม Defects4J
Overall denominator	854 bugs

ส่วน	Configuration
Environment	Ubuntu 22.04 Docker; Java 11 default; timezone America/Los_Angeles

ค.3 โครงสร้าง Evaluation Summary

```
{
  "project": "<Project>",
  "bug_id": "<Bug ID>",
  "tool": "<evosuite|kex|gemini|claude>",
  "generated_files": <int>,
  "candidate_tests": <int>,
  "classification_counts": {...},
  "bug_revealing_tests": <int>,
  "bug_detected": <true|false>,
  "generation_status": "<status>",
  "evaluation_status": "<success|compile_failed>"
}
```

ค.4 โครงสร้าง Coverage Summary

```
{
  "project": "<Project>",
  "bug_id": "<Bug ID>",
  "tool": "<tool>",
  "status": "success",
  "prepared_files": <int>,
  "lines_total": <int>,
  "lines_covered": <int>,
  "conditions_total": <int>,
  "conditions_covered": <int>,
  "line_coverage": <float>,
  "condition_coverage": <float>,
  "has_failing_tests": <true|false>,
  "failing_test_count": <int>
}
```

ค.5 ไฟล์ Final Aggregate

- master_results.csv — ผลระดับ Bug x Technique
- technique_summary.csv — ผลรวมหลักของ 4 เทคนิค
- evaluation_status_counts.csv และ coverage_status_counts.csv — การกระจายสถานะ
- common_evaluation_summary.csv และ common_coverage_summary.csv — Same-Case Comparison
- project_summary.csv — ผลแยกตาม Defects4J Project

- ai_token_per_bug.csv, ai_token_summary.csv, ai_token_common_summary.csv — Token Analysis
- evaluation/final/charts/ — กราฟผลการทดลอง

ข้อมูลในภาคผนวกนี้มีวัตถุประสงค์เพื่อให้ผู้อ่านเชื่อมโยงผลในรายงานกับ Artifact ใน Repository และสามารถตรวจสอบหรือทำซ้ำ Pipeline ได้อย่างเป็นระบบ